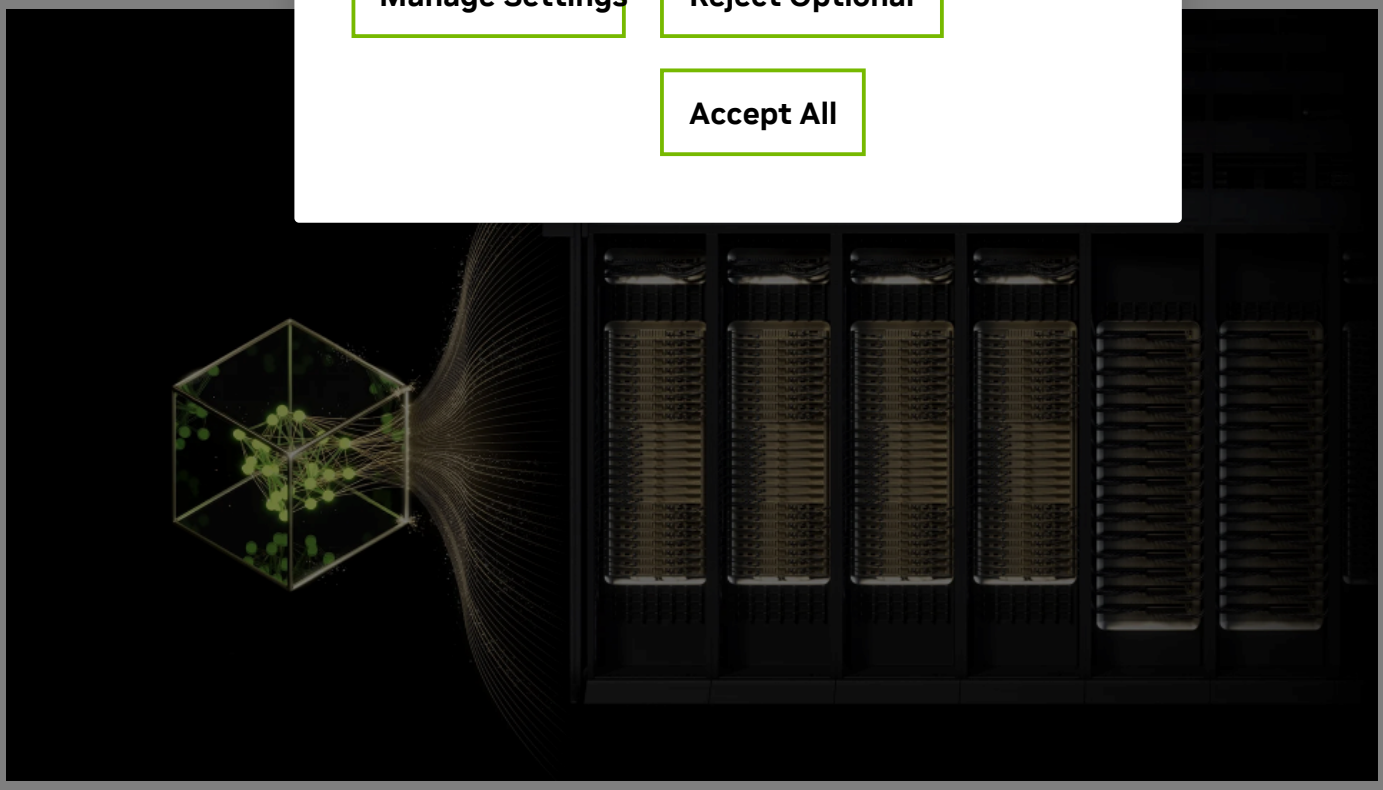
NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Manage Settings Reject Optional

Accept All

Developer Tools &

Enhan Inferer the NV Library h sfer



Mar 09, 2026

+43 Like Discuss (0)

By Seonghee Lee, Moein Khazraee, Timothy Stamler, Adit Ranadive and Chris Hoge

AI-Generated Summary

more users while reducing latency. Distributed inference frameworks use techniques such as disaggregated serving, KV cache loading, and wide expert parallelism.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

In disaggregated ser ate GPUs, requiring efficient K hput communication to m egated serving.

In KV cache loading, d agentic AI workloads such as c V, the previous results can mputing them as prefill. This art of inference workloads

In wide expert parall iate results (activations) have to e requirement for ultra ween stages, these transfers are t ed to as device side APIs for networking, or device API in short.

Another unique feature of inference workloads is their need for dynamicity and resiliency. Services can run 24 hours a day, seven days a week. While based on user demand, the number of GPUs used can change. There can also be more fine-grained dynamicity. The ratio of GPUs doing prefill and decode might change or, in the case of elastic expert parallelism, the number of replicated experts or even total experts can change.

In the event of failures, the system needs to be resilient, running at lower throughput for a brief period of time until the recovery mechanism handles the failure. This requirement extends the system's dynamicity needs by detecting failures and managing the transitional state until recovery completes.

Finally, while there is a need for heterogeneous hardware support in terms of memory and storage, there can be heterogeneity in compute hardware as well. Handling each of these unique hardware components can become cumbersome. This requires a library that can unify different communication and storage technologies, which ensures that frameworks can efficiently move

NVIDIA Inference Transfer Library (NIXL) is an open source, vendor-agnostic, data movement library designed to s fering a unified and powerfu echnologies.

This post explains N adata exchange, descripto also explains the usage flow of th few examples to help you

What is N

NIXL is an open sour ference frameworks. NIXL pr variety of data transfer challer formance.

This API supports m GPU-Direct storage, block and file storage, and advanced cloud storage options including S3 over RDMA and Azure Blob Storage. It is vendor-agnostic and can run across diverse environments. For example, it supports Amazon Web Services (AWS) with EFA networking and Trainium or Inferentia accelerators, as well as Azure with RDMA networking. The team is working with Google Cloud to add both RDMA and GPUDirect-TCPXO networking. NIXL is already a key component of many AI inference frameworks such as NVIDIA Dynamo, NVIDIA TensorRT LLM, vLLM, SGLang, Anyscale Ray, LMCache, and more.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Figure 1. NIXL addr *s resources,*

Core use cases of NIXL include:

- **Disaggregation**: Moves KV blocks between prefill and decode workers with high throughput and low latency
- **Long context KV cache storage**: Stores KV cache data in some long term storage medium to avoid recomputation later
- **Weight transfer**: Ships model weights to GPU nodes for fast startup or resharding. The weights might come from GPU memory, host memory, or storage
- **Reinforcement learning**: Streams updated weights from learners to actors with minimal transfer overhead
- **Elastic expert parallelism**: Dispatches and combine stages in expert parallelism can be done through NIXL, with support for dynamic reconfigurations

The unified NIXL API is for different types of memory and storage, while its pluggable backend design allows this API to target many different high-performance technologies (RDMA, GPU-initiated networking, GPU-Direct storage, NVMe, Object Stores, and so on). NIXL is designed to have a fully non-blocking API and incur minimal overhead. This enables efficient overlapping of communication and computation with high-performance zero-copy transfers.

nodes are added based on user load, removed due to failures, or recycled for different purposes all the time.

These features enab r the user of the NIXL library, whil nds. Additionally, dynami eted for inference applicatio

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NIXL des

NIXL functions as a ous network and storage backen nines when transfers are require is is done in an object-oriented m ng from a remote agent (or wit eferred to as put and get.

This terminology enables a unified API that supports both efficient one-sided network communications and storage transfers. The user describes any memory or storage through a list of descriptors, which has an encompassing type to indicate if the data is stored in host memory, GPU memory, or some type of storage. Each descriptor within a descriptor list points to a location in memory or storage. For example, some base address and a size on a host memory, GPU, or SSD, or similarly a location within a file or storage object. Note that each set of transfer descriptors must be from the same memory type but can transverse memory types across the transfer. For example, sending from GPU memory to host memory.

The conductor gives the NIXL agent access to the desired allocated memories through a registration call. When using one-sided read or write operations, keys or identifiers are generated, so only other processes that have the proper key can access that memory. NIXL encapsulates such information for these registrations, as well as the required connection info, into a metadata object. Inside the NIXL agent, Memory Section and Metadata Handler components are in charge of managing the necessary local and remote information respectively.

NVIDIA DEVELOPER Join

process can directly obtain the metadata object from one agent, and load it into another agent. For the case of device API usage by GPU kernels, there is one more preparation step necessary to send the relevant

This metadata excha ocal memory or storage transfers. uted storage system, becoming a l facilitating methods to exchang al metadata service such as etcd

Now the conductor NIXL first checks whether the nductor process can ask the er status until it is complete, in a n ner, from the GPU kernel.

The NIXL agent will i r request, and deliver the prep sired backend). This enables NIXL to achieve high performance and remain hardware agnostic. Figure 2 shows the current list of supported backends, which is expanding with the rapid adoption of NIXL.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Figure 2. NIXL architecture consists of a core transfer agent with a Memory Section and Metadata Handler, and supports multiple transfer backend plugins through an API

The following NIXL u use the NIXL API to perform an as ance networking library.

For the case of trans itiator, which creates and starts th e target, whose memory is be

These roles are defin es the operation. The initiat ds a notification to the ta

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Setting up

Setting up the initiat

Step 1: Agent

At startup, each application spawns a runtime agent configured with relevant initialization parameters. The agent initializes the transfer backends specified or uses UCX as default if none are provided. UCX is a community-driven networking library and is widely tested internally. The user also gives a name to the agent, which can be any string, such as an UUID.

Step 2: Memory registration

Users allocate memory on their chosen devices—GPU, CPU, storage—and register these regions with the agent through NIXL descriptors. NIXL will internally pass that information to each relevant backend that supports that memory type.

Optimization tip: Most backend registrations must go through a kernel call, which can be time consuming. It is advised to minimize the number of registrations by registering larger blocks of memory, as transfers can be created anywhere within the registered memory.

Step 3: Metadata exchange

that enables dynamicity for the NIXL library.

Optimization tip: Wh tadata needs to be exchanged. If c e other side has fixed buffers to tor agent. This removes the ne

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Preparing a

After the metadata l rforms the following steps:

Step 1: Create

The transfer request itiator and the target descripto ill verify these descriptors, d at backend if preparations are req

Step 2: Start (or post) the transfer request

NIXL issues this request to the appropriate backend, making this low overhead. The backend performs the data transfer between the source and destination addresses. The backend library performing the transfer uses the system libraries and drivers underneath to perform the transfer efficiently.

Step 3: Check transfer status

To enable overlap of compute and communication, the post call is nonblocking, which requires the user to check the status of a transfer separately. Note that the transfer might complete, or might result in an error (network failure, for example). Such failure does not impact the other agents in the system, nor the transfers within the same agent that don't face that network failure.

On the target side, the user can look for notifications that indicate a transfer was complete. The

Tear down

When a NIXL agent i d memories.
If an active transfer n an error
status. If local transf destruction.
However, it is advise

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NIXL per ols

Performance benchr sed to verify
that a system is ope rprise
system. They can als d.

NIXL provides a two- and an LLM-
aware profiler called

NIXLBench is intentionally model-agnostic and maintains a simple system view. It executes real data transfers, sweeps block and batch sizes, and reports bandwidth metrics with latency percentiles. NIXLBench relies on etcd to exchange transfer metadata for network backends, but not for storage backends as there is no need for metadata exchange.

KVBench offers significant advantages for LLM engineers by accelerating benchmarking and iteration through the automatic calculation of exact KV cache I/O size and batch size for supported models, and generates a ready-to-run NIXLBench command. KVBench can also instantiate profiling of KVCache transfers using its CTPerfTest module.

Get started with NVIDIA Inference Transfer Library

NIXL software is fully open source and available on the ai-dynamo/nixl GitHub repo. It is written in C++ for high performance, efficiency, and composability. Several bindings are available, including C, Python, and Rust.

inference frameworks and workloads.

To learn more, you ca starting point, basic_two_pe metadata exchange, a single R ion, expanded_two_peer Ds and WRITEs with various d usage of patterns in notificati

We welcome questic unity on GitHub. Stay tuned f , check out these additional resc

- NIXL core concept
- NIXL Python API u
- NIXL telemetry sy
- NIXL Benchmark
- KVBench workflow

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Acknowledgments

The NVIDIA Inference Transfer Library product team acknowledges the valuable contributions of all open source developers, contributors, testers, and community members who have participated in its evolution.

Discuss (0) +43 Like

Tags

Developer Tools & Techniques | MLOps | Networking / Communications | HPC / Scientific Computing | Dynamo | NVLink | TensorRT-LLM | Intermediate Technical | Deep Dive | Tutorial | AI Agent | AI Inference | Inference Performance | Python

About the Authors

NVIDIA, focusing on AI Inference-related products. Seonghee holds a master's in computer science from Stanford University and a bachelor's joining al-time AI

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

architect at o-end y a software and At NVIDIA, rchitecture of ink.

About Timothy Stamler

Timothy Stamler is a senior systems architect for the Network Architecture team at NVIDIA and works on a variety of systems and performance optimization problems. Tim did his PhD at UT Austin, focusing on optimizing operating system IO stacks and making storage and network stacks work more cooperatively. Since coming to NVIDIA, he's worked on extending the RDMA verbs API and making it easier for AI inference platforms to utilize high speed network technology through libraries like NIXL.

View all posts by Timothy Stamler >

About Adit Ranadive

Adit Ranadive serves as senior software architect in the NVIDIA Networking Software Advanced Development Group, focusing on system technologies that enhance AI inference workloads. His work includes enabling high-performance computing software to utilize DPUs for IO stream processing and advancing distributed computing architectures.

View all posts by Adit Ranadive

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

hnical
open source
nance
n Applied
or's degree in
rsity in St.

Comments

Start the d

Related posts

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Smart Multi-Node Scheduling for Fast and Efficient LLM Inference with NVIDIA Run:ai and NVIDIA Dynamo

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Reducing Colc

Streamer

un:ai Model

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NVIDIA Dynamo Accelerates llm-d Community Initiatives for Advancing Large-Scale Distributed Inference

The Next Generation of AI Begins

It all starts here. June 1–4 | Taipei

Register Now

NVIDIA GTC

Sign up for NVIDIA News

Subscribe

Follow NVIDIA Developer

NVIDIA DEVELOPER

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.